

Mojo: An MLIR-Based Language for HPC

Yannick Schürmann

Abstract—Mojo is a new systems language built on MLIR...

I. INTRODUCTION

As GPU hardware diversifies and AI workloads grow alongside traditional HPC simulations, the demand for portable, high-performance software has never been greater. Yet the tools available to meet that demand force developers into an uncomfortable tradeoff between productivity and performance.

This tradeoff stems from two concrete problems. The first is the two-language split, where HPC code is typically prototyped in Python for ease of use, then rewritten in C++ or CUDA to meet performance requirements [1,2]. NumPy, for instance, is little more than a Python interface over hand-optimized C extensions. The second is vendor lock-in, where most high-performance GPU code targets NVIDIA’s CUDA ecosystem exclusively [3]. As HPC clusters increasingly incorporate hardware from AMD, Intel, and others, software written for one vendor’s stack may require substantial effort to port, or simply fail to run elsewhere. The Frontier supercomputer, the first exascale machine, illustrates this directly: it is built entirely on AMD GPUs, meaning software written for NVIDIA CUDA does not run on it without significant porting effort [4].

Together, these two problems demand expertise across multiple ecosystems, increase maintenance burden, and slow the path from research to production. No existing solution addresses both simultaneously. Julia eliminates the language split through just-in-time (JIT) compilation to native performance, but remains vendor-dependent [5]. Kokkos and SYCL offer hardware portability but stay firmly within the C++ ecosystem, leaving the language split intact [6,7]. Solving both problems simultaneously would allow HPC developers to write productive, high-performance code in a single language, without sacrificing portability across hardware vendors.

MLIR (Multi-Level Intermediate Representation) is a compiler infrastructure designed to operate across multiple abstraction levels simultaneously, rather than targeting one fixed level like LLVM IR (Intermediate Representation) [8]. A matrix multiplication, for instance, can be expressed as a high-level linear algebra operation and progressively lowered through loop optimizations and memory layout decisions down to vendor-specific GPU instructions.

Mojo, built directly on MLIR, is the first credible candidate to address both problems [9]. It extends Python with

systems programming features including ownership, static typing, and compile-time metaprogramming. These are necessary for MLIR to do its job. Without static types the compiler cannot lower through abstraction levels, without ownership there is no way to manage GPU memory without a garbage collector, and without compile-time metaprogramming problem sizes and layouts remain unknown until runtime, preventing the specialization MLIR depends on. Together these properties allow Mojo to compile high-level, vendor-agnostic code to optimized, vendor-specific GPU instructions. This paper examines Mojo’s design and evaluates its HPC viability through benchmarking against established GPU programming models.

We begin with the background on LLVM IR and MLIR, establishing the compiler foundation Mojo builds on. We then take a look at prior approaches to the two problems and where each falls short. From there we cover how Mojo is implemented as a set of MLIR dialects and what that means for language features and hardware portability. Finally we go through real HPC benchmarks on NVIDIA and AMD GPUs, discuss what the results tell us about Mojo’s current strengths and gaps, and consider its future trajectory.

II. BACKGROUND

To understand the design principles behind Mojo and the improvements it brings, it is first essential to understand the compiler infrastructure it is built on. Mojo is implemented directly on top of MLIR, which itself builds on the original LLVM IR. Understanding how IR works, where it falls short, and how MLIR addresses those shortcomings is what gives context to Mojo’s key properties, which allow it to elegantly address both the two-language split and vendor lock-in.

A. LLVM [10]

LLVM is a compiler framework designed to normalize code representation across languages, making it possible to apply a shared set of optimizations to any code compiled with it [10]. To achieve this, LLVM represents programs in Static Single Assignment (SSA) form, where each variable is defined exactly once. This representation, called LLVM Intermediate Representation (LLVM IR), abstracts away language-specific constructs and reduces code to raw semantics, allowing the same optimization passes to work across C, C++, Rust, Swift, and many other languages. IR is often described as roughly C with vectors [8], operating at a single fixed abstraction level. This means both high-level language features and machine-specific details are out

of scope. To illustrate, consider the matrix multiplication below.

```
for (int i = 0; i < N; i++)
  for (int j = 0; j < N; j++)
    for (int k = 0; k < N; k++)
      C[i][j] += A[i][k] * B[k][j];
```

Compiled to IR, the loop structure disappears entirely into control flow. What remains is raw pointer arithmetic operating on individual memory addresses. Each % assignment is an instance of SSA. The snippet below shows only the inner loop body for brevity, as the full IR is significantly longer.

```
%a_p = getelementptr float, float* %A, i64 %a_i
%a_v = load float, float* %a_p
%b_p = getelementptr float, float* %B, i64 %b_i
%b_v = load float, float* %b_p
%prod = fmul float %a_v, %b_v
%c_p = getelementptr float, float* %C, i64 %c_i
%c_v = load float, float* %c_p
%sum = fadd float %c_v, %prod
store float %sum, float* %c_p
```

the compiler can optimise individual operations but has lost the information that this is a matrix multiply, what the memory layout is, or that the loops are parallelisable

As stated in the original LLVM paper, LLVM is not intended to be a universal compiler IR and does not represent high-level language features directly, nor does it capture machine-dependent features or code sequences used by back-end code generators [10]. This works well for CPU-targeted, homogeneous compilation but becomes a limiting factor when targeting heterogeneous hardware or domain-specific optimizations. Many problems are simply better modeled at a different abstraction level, for example source-level analysis of C++ code is very difficult on IR [8]. As a result, languages like Swift, Rust, and Julia each ended up building their own mid-level IRs before targeting LLVM, leading to fragmented and duplicated compiler infrastructure with each team reimplementing the same class of optimizations [8], as illustrated in Figure 1.

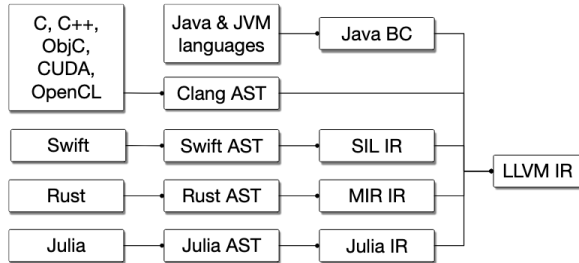


Figure 1: Compilation pipeline of different languages with multiple mid-level IRs for language-specific optimization with common backend for multiple hardware targets [8]

B. MLIR [8]

1) Motivation

The fixed abstraction level of LLVM IR, while powerful for general-purpose compilation, created a recurring problem. Whenever engineers needed to solve a domain-specific compiler challenge, they had no good place to put it. Problems that were too high-level for LLVM IR and too low-level for the source language led engineers to develop their own intermediate compilers, resulting in fragmented and duplicated infrastructure across the industry. This fragmentation was particularly severe in ML and HPC compiler stacks. MLIR was developed to solve this by making it cheap to define new abstraction levels within a single, shared compiler infrastructure [8].

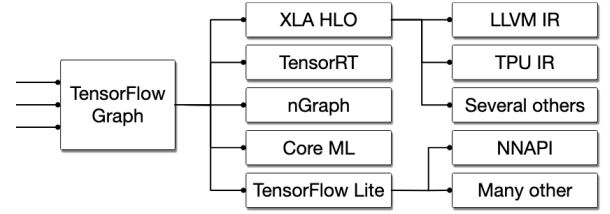


Figure 2: Fragmented TensorFlow stack [8]

Frameworks like TensorFlow were composed of many different compilers, graph technologies, and runtime systems with no shared infrastructure [8]. Each component was developed independently, with no common design point or shared best practices in compiler construction. This fragmentation is illustrated in Figure 2. The end user could experience the consequences of this fracturing directly, through poor error messages, unpredictable performance, and difficulty generalizing the stack to support new hardware [8].

2) Core Idea

At the core of MLIR is the principle of being multi-level, where multiple abstraction levels can coexist within a single framework. This is captured by MLIR’s guiding design philosophy of “little builtin, everything customizable” [8]. The system is built on a minimal set of fundamental concepts, namely types, operations, and attributes, with most of the intermediate representation left open for extension. This customizability ensures the system can adapt to changing requirements and remain applicable across future problems [8]. A key measure of success for this approach is the ability to express a diverse set of abstractions, from machine learning graphs and abstract syntax trees all the way down to instruction-level representations like LLVM IR, all without hardcoding any of these concepts into the system [8]. This stands in direct contrast to LLVM, which provided only a single fixed abstraction level [8]. By making abstraction levels customizable, MLIR allows compiler engineers to work at the right level of detail

for their problem, without building an entirely separate compiler infrastructure to do so.

3) Dialects

Dialects are the primary mechanism through which MLIR achieves its extensibility. A dialect is a logical grouping of operations, types, and attributes under a unique namespace, and adding a new dialect is how compiler engineers introduce new abstractions into the system [8]. The namespace appears as a dot-separated prefix on each operation, as can be seen in Figure 3 below.

```
linalg.matmul
  ins(%A, %B : memref<?x?xf32>,
      memref<?x?xf32>)
  outs(%C : memref<?x?xf32>)
```

Figure 3: Matrix multiplication expressed in MLIR’s *linalg* dialect, a single operation in the high-level *linalg* namespace.

The separation into dialects is modular by design, and operations from different dialects can coexist at any level of the IR at any time, allowing them to be mixed and progressively lowered [8]. This is what makes it possible to represent a matrix multiplication at the high-level *linalg* dialect, lower the loop structure to the *affine* dialect, then lower further to *scf* (structured control flow), and finally to LLVM IR, each step preserving only what is still needed.

This composability also enables reuse across domains. An OpenMP dialect, for example, can be shared between Fortran and C language frontends rather than each implementing their own parallel constructs [8]. Similarly, hardware vendors can contribute target-specific dialects such as NVVM for NVIDIA GPUs or AMDGPU for AMD, making hardware portability a property of the compiler infrastructure rather than something each language or framework has to solve on its own.

4) Progressive Lowering

Progressive lowering is the principle that a high-level representation should be brought down to hardware-specific instructions not in a single step, but gradually through a series of small transformations at multiple abstraction levels [8]. The need for this stems from the variety of platforms and programming models a generic compiler infrastructure has to support, as the same source program may need to target a CPU, a GPU, or a custom accelerator, each requiring different lowering paths.

To see what this looks like in practice, consider the matrix multiplication example again. In MLIR, this can be expressed at the high-level *linalg* dialect and then progressively lowered through *affine* (loop structure), *scf* (structured control flow), *cf* (control flow), and finally to LLVM IR for code generation, as shown in Figure 5.

This stands in contrast to rigid compiler pipelines like Clang, which lowers from ASTs to LLVM IR, to SelectionDAG, to MachineInstr, and to MCInst in a fixed sequence where compiler passes can only operate at one level at a time [8]. The key property that sets MLIR apart is that different parts of the IR can sit at different abstraction levels simultaneously. As the paper describes, “mixing different levels of abstractions and different concepts in the same IR is a key property of the system to allow a part of the representation to remain in higher-level abstraction while another part is lowered” [8]. This means a compiler for a custom accelerator can reuse high-level loop structure alongside primitive scalar/vector instructions specific to that hardware, all within the same IR at the same time.

5) SSA and Nested Regions

MLIR extends SSA with *nested regions* as a first-class concept. In MLIR, operations can contain regions, which in turn contain blocks and further operations, forming a recursive structure. This allows higher-level abstractions like loops, closures, and parallel constructs to live directly in the IR at any abstraction level [8]. This structure is illustrated in Figure 4.

To see what this means for our running example, consider the same matrix multiplication expressed in MLIR’s *affine* dialect. The loop structure that disappeared in LLVM IR is fully preserved here as nested *affine.for* regions, each with its own SSA values and control flow.

```
affine.for %i = 0 to %M {
  affine.for %j = 0 to %N {
    affine.for %k = 0 to %K {
      %a    = affine.load %A[%i, %k]
              : memref<?x?xf32>
      %b    = affine.load %B[%k, %j]
              : memref<?x?xf32>
      %c    = affine.load %C[%i, %j]
              : memref<?x?xf32>
      %prod = arith.mulf %a, %b : f32
      %sum  = arith.addf %c, %prod : f32
      affine.store %sum, %C[%i, %j]
              : memref<?x?xf32>
    } } }
}
```

Figure 5: Matrix multiplication after lowering from the single *linalg* operation in Figure 3 to the *affine* dialect. The loop structure is preserved as nested regions with affine semantics.

The loop body lives inside the operation as a region rather than as a separate function. The compiler can therefore analyse and transform the loop structure, for tiling, parallelisation, or vectorisation, without losing the surrounding context [8].

```

%results:2 = "d.operation"(%arg0, %arg1) ({
  // Regions belong to Ops and can have multiple blocks.
  ^block(%argument: !d.type):
    // Ops have function types (expressing mapping).
    %value = "nested.operation"() ({
      // Ops can contain nested regions.
      "d.op"() : () -> ()
    }) : () -> (!d.other_type)
    "consume.value"(%value) : (!d.other_type) -> ()
  ^other_block:
    "d.terminator"() [^block(%argument : !d.type)] : () -> ()
})
// Ops can have a list of attributes.
{attribute="value" : !d.type} : () -> (!d.type, !d.other_type)

```

Figure 4: Operation (Op) is the main entity in MLIR. Operations contain a list of regions, regions contain a list of blocks, blocks contain a list of Ops, enabling recursive structures [8].

6) Why This Matters for Portability

The properties described above come together to directly enable hardware portability. Because MLIR’s abstraction levels are defined by dialects, the same high-level representation can be lowered to different hardware backends simply by swapping the lowering dialect. A matrix multiplication expressed at the *linalg* level can be lowered to NVVM for NVIDIA GPUs or AMDGPU for AMD without any changes to the source representation, keeping the code hardware-agnostic at the top and hardware-specific only at the final lowering step. This is the architectural foundation that allows Mojo, as the first language built directly on MLIR, to inherit vendor portability as a compiler property rather than a library concern.

III. PRIOR WORK

Several approaches have been developed to address vendor lock-in and the two-language problem in HPC. None of them fully solve both at once.

A. OpenCL [11]

- first open standard for GPU programming across vendors (NVIDIA, AMD, Intel)
- addresses vendor lock-in but exposes a low-level C API with significant boilerplate
- kernel code is still written in C, so the two-language problem is not addressed
- largely superseded in practice by vendor-specific toolkits and higher-level successors

B. SYCL [7]

- modern C++ successor to OpenCL with much better ergonomics
- keeps the vendor portability of OpenCL
- the language is still C++, so the gap between Python-based science workflows and kernel code remains

C. Kokkos [6]

- C++ library that abstracts over backends including CUDA, HIP, and OpenMP

- solid vendor portability story with a mature ecosystem
- requires C++, so the two-language split with Python is not addressed

D. Julia [1,5]

- high-level JIT-compiled language that reaches near-native performance, directly targeting the two-language problem
- GPU support exists via packages like CUDA.jl and AMDGPU.jl, but these are vendor-specific backends, so vendor lock-in is not resolved as cleanly as in the C++ approaches above

IV. MOJO: BUILT ON MLIR

A. Mojo as an MLIR-native language

- Mojo is the first programming language implemented as a set of MLIR dialects, not a language that targets MLIR from outside [8]
- Mojo inherits these properties directly as language features rather than leaving them as compiler internals
- this is possible because of MLIR’s design principle of “little builtin, everything customizable” [8]: the IR has no fixed set of operations or types, so an entire language can be implemented as a set of dialects rather than requiring a separate compiler

B. Progressive lowering as a language feature

- performance in Mojo is controlled through type annotations on **def** functions
- the more type information provided, the further the compiler can lower through MLIR dialects toward hardware-specific operations
- a **def** with no annotations behaves dynamically like Python, while the same **def** with type annotations compiles to typed MLIR ops and then to native code
- this makes MLIR’s principle of “maintain structure of computation and progressively lower to the hardware abstraction” [8] user-facing: no rewrite into a separate language is required

C. Compile-time specialization via parameters

- Mojo distinguishes compile-time values (parameters, written in square brackets []) from runtime values (arguments, written in parentheses ())
- “In Mojo, ‘parameter’ always means a compile-time value, and ‘argument’ always means a run-time value” [12]
- “the compiler resolves the parameter values during compilation, and creates a concrete version of the function for each unique parameter value” [12]
- this exposes MLIR’s constant folding and specialization directly, replacing C++ template metaprogramming with Python-like syntax

D. Value types and memory layout via *struct*

- **struct** defines value types with explicit memory layout, mapping directly to MLIR’s type system
- “because the compiler knows every field’s type at compile time, it can calculate the struct’s exact memory footprint” [12]
- “Python classes are dynamic: they allow for dynamic dispatch, monkey-patching, and dynamically binding instance fields at runtime. Mojo structs are static: they are bound at compile-time” [12]
- this static layout is necessary for predictable GPU memory access patterns

E. Python interoperability

- Mojo is designed to be a superset of Python: the goal is that any valid Python program is also a valid Mojo program, with Mojo adding opt-in performance and type safety on top [12]
- “not only does Mojo use a Pythonic syntax, the plan is to provide full compatibility with the Python ecosystem” [12]
- existing Python libraries like numpy and scipy can be imported and used directly without any FFI boilerplate [12]
- because MLIR allows “mixing different levels of abstractions and different concepts in the same IR” [8], hot paths can be lowered to native code while the rest of the program remains in Python

F. Hardware portability via dialects

- adding a new hardware target in Mojo reduces to adding a lowering dialect in MLIR, for example NVVM for NVIDIA GPUs and AMDGPU for AMD GPUs
- vendor portability is a compiler property rather than a library abstraction
- the same Mojo source compiles to different backends without code changes, directly realising MLIR’s goal of “improving compilation for heterogeneous hardware” [8]

V. MOJO IN HPC: APPLICATION AND EVALUATION

A. GPU Programming Model

- CUDA-like kernel syntax: `block_idx`, `thread_idx`, `block_dim`, `DeviceContext` for memory allocation and kernel dispatch
- `LayoutTensor` carries layout in its type statically for MLIR to optimise
- “performance-critical information such as problem and block sizes, array layout, and variable types must be well-defined at compile time” [9]
- “unlike most HPC software applications, in which much of the information is known only at run time (i.e., compile once, run many times), the reliance on MLIR optimizations restricts Mojo programs to providing performance-critical information at compile time. Although this is desired for JIT approaches (e.g., Julia), it can be a challenge when deploying code that has access to only certain information at run time (e.g., real-time workflows, adaptive mesh codes, statistical random walks)” [9]
- Python interop is runtime-only via CPython, not compiled through MLIR. “Mojo introduces this fragmentation in the language by keeping a clear separation between (i) JIT- or AOT-MLIR compiled high-performance code and (ii) Python interoperability as a run-time-only entity” [9]
- memory safety via value ownership: “heap-allocated values have exactly one owner at a time. When a value’s lifetime ends, Mojo automatically deallocates it by calling its destructor, thus eliminating the need for a garbage collector” [9]
- “Mojo strives for zero-cost abstraction so that higher level abstractions simply compile down to fast, clean low-level programming languages without incurring runtime bloat” [13]

B. Benchmarking

Godoy et al. [9] benchmark four HPC proxy kernels against CUDA (NVIDIA H100) and HIP (AMD MI300A).

1) Memory-Bound Kernels

- seven-point stencil: 82–87% of CUDA on H100. “Mojo kernels use more registers-per-thread (24) than CUDA (21) for the same operation, which explains the performance differences in L-caches and kernel duration.” On par with HIP on MI300A. $\Phi = 0.92$ [9]
- BabelStream: simple array ops slightly outperform CUDA on H100. “Mojo is highly optimized for simple 1D array operations in BabelStream and even slightly surpassed CUDA’s performance thanks to fewer cached memory operations.” Dot reduction at 78% of CUDA. Indistinguishable from HIP on MI300A. $\Phi = 0.96$ [9]

2) Compute-Bound Kernels

- miniBUDE: “significant sensitivity to fast-math compilation, which results in noticeable performance gaps in the Mojo implementation.” Only ~38% of HIP on MI300A. $\Phi = 0.54$ [9]
- Hartree-Fock: $2.5\times$ faster than CUDA at small sizes (187 ms vs 472 ms at 256 atoms), $55\times$ slower at 1024 atoms. On AMD, “atomic operations must be further analyzed and better understood given Mojo’s peculiar results of overperforming CUDA and underperforming HIP.” $\Phi = 0.92$ misleading [9]

3) Overall

- “Mojo’s performance is competitive with CUDA and HIP for memory-bound kernels, whereas gaps exist on AMD GPUs for atomic operations and for fast-math compute-bound kernels on both AMD and NVIDIA GPUs” [9]

C. Current Scope and Limitations

- single-node only: “Mojo’s interoperability with MPI is an open question given that Python’s mpi4py is well-established and Python’s interoperability is orthogonal to the compile-time nature of Mojo” [9]
- low-level ecosystem: “Mojo is still a low-level programming alternative to NVIDIA’s CUDA and AMD’s HIP. Many of the HPC codes rely on interactions with highly optimized high-level libraries, and support for this functionality (e.g., portable BLAS operations) is still being defined” [9]
- tooling gaps: “debugging Mojo code is only supported by NVIDIA’s Nsight Compute CLI (ncu) or the CUDA-GDB debugger... we were unable to find an officially supported tool to successfully profile with AMD’s rocprof” [9]

VI. DISCUSSION

A. The Language Split: Is Mojo as Productive as Claimed?

- the “Python-like ease with C++ performance” pitch conflicts with what GPU programming in Mojo actually looks like: explicit thread indexing, compile-time sizes, manual memory layouts. “The learning curve and programming requirements are still fairly low-level” [9]
- Akre et al. note a “steep learning curve for developers accustomed to Python’s higher-level abstractions” once Rust- and C++-inspired safety features are introduced [13]
- Python interoperability is runtime-only and not compiled through MLIR, meaning the high-productivity Python ecosystem and the high-performance Mojo world remain separate in practice, reproducing the split Mojo set out to close
- “Mojo strives for zero-cost abstraction so that higher level abstractions simply compile down to fast, clean low-level programming languages without incurring

runtime bloat” [13], but this only holds for the compile-time side of the language

B. Vendor Lock-in: Is Portability Worth the Performance Cost?

- for memory-bound workloads the trade-off is favourable: one codebase delivers around 87% of CUDA on NVIDIA and around 100% of HIP on AMD, “competitive with CUDA and HIP for memory-bound kernels” [9]
- for compute-bound workloads the case is weaker: $\Phi = 0.54$ for miniBUDE means single-source portability comes at a significant cost on both vendors
- compared to Kokkos and SYCL, which also incur portability costs, Mojo is less mature and lacks the high-level library ecosystem HPC codes depend on: “support for this functionality (e.g., portable BLAS operations) is still being defined” [9]
- unlike SYCL and Kokkos, Mojo’s portability is a compiler property rather than a library abstraction: adding a new hardware target means adding a dialect, not rewriting user code, an architectural advantage that may compound as the ecosystem matures

C. Can It Get Better?

- the performance gaps are implementation gaps, not architectural ones. **fast-math** and immature atomic operations on AMD are known missing features, not fundamental MLIR limitations: “we look forward to this capability being available for these workloads in the near future” [9]
- “Mojo might present an opportunity to explore future compiler optimizations via MLIR through AOT or JIT” as the language matures [9]
- “our expectation is that Mojo will continue to evolve under this programming paradigm, with the promise of becoming open-source by 2026, as it moves toward becoming a viable HPC + AI programming language” [9]
- the MPI story is a harder open question: Python interoperability is runtime-only and orthogonal to Mojo’s compile-time model, making multi-node HPC integration structurally unclear [9]

D. Overall Assessment

- Mojo is the only approach that structurally addresses both the language split and vendor lock-in simultaneously. Julia solves the language split better today in terms of ecosystem maturity, Kokkos and SYCL solve vendor lock-in better today in terms of library coverage
- “unlike other programming models that attempt to speed up Python or compile to LLVM from the ground up (e.g., Julia, Rust), Mojo closes performance gaps by offering a superset language that combines rich compile-time metaprogramming via

MLIR, performance-portable GPU programming, and run-time interoperability with the Python ecosystem” [9]

- “structurally sound” and “practically ready” are different things. Mojo’s bet is that building on MLIR will compound over time in ways library-based alternatives cannot match, but this remains to be demonstrated at scale

VII. FUTURE OUTLOOK

A. Licensing and Openness

- Mojo is developed by Modular Inc. and is currently proprietary: source-available but not open source
- this creates a vendor dependency risk for HPC adoption, as institutions may be reluctant to build workflows around a closed language
- in contrast, the alternatives discussed in prior work (SYCL, Kokkos, Julia) are all fully open source with active community governance
- Godoy et al. note “the promise of becoming open-source by 2026” [9], which could accelerate adoption significantly if realised

B. Maturity

- Mojo is still pre-1.0. The language specification, standard library, and GPU backend are under active development and subject to breaking changes
- features critical for HPC such as multi-node support, MPI interoperability, and broader hardware targets beyond NVIDIA are not yet mature or are absent entirely

C. Looking Ahead

- a maturing ecosystem including a package manager, debugger tooling, and broader MLIR dialect support would reduce the friction that currently makes Mojo a research interest rather than a production choice
- the MLIR foundation gives Mojo a structural advantage: adding support for a new hardware target reduces to writing a new lowering dialect, which is cheaper than the library-level rewrites required by SYCL or Kokkos
- the open question is whether the ecosystem and governance will catch up to the technical promise

VIII. REFERENCES

- [1] J. Bezanson, A. Edelman, S. Karpinski, and V. Shah, “Julia: A Fresh Approach to Numerical Computing.” Accessed: Apr. 28, 2026. [Online]. Available: <https://epubs.siam.org/doi/epdf/10.1137/141000671>
- [2] J. Bezanson, S. Karpinski, V. Shah, and A. Edelman, “Why We Created Julia.” Accessed: Apr. 28, 2026. [Online]. Available: <https://julialang.org/blog/2012/02/why-we-created-julia/>
- [3] J. Nickolls, I. Buck, M. Garland, and K. Skadron, “Scalable parallel programming with CUDA,” in *ACM SIGGRAPH 2008 classes*, in SIGGRAPH ’08. New York, NY, USA: Association for Computing Machinery, Aug. 2008, pp. 1–14. doi: [10.1145/1401132.1401152](https://doi.org/10.1145/1401132.1401152).
- [4] S. Atchley et al., “Frontier: Exploring Exascale,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, in SC ’23. New York, NY, USA: Association for Computing Machinery, Nov. 2023, pp. 1–16. doi: [10.1145/3581784.3607089](https://doi.org/10.1145/3581784.3607089).
- [5] T. Besard, C. Foket, and B. D. Sutter, “Effective Extensible Programming: Unleashing Julia on GPUs,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 4, pp. 827–841, Apr. 2019, doi: [10.1109/TPDS.2018.2872064](https://doi.org/10.1109/TPDS.2018.2872064).
- [6] C. R. Trott et al., “Kokkos 3: Programming Model Extensions for the Exascale Era,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 805–817, Apr. 2022, doi: [10.1109/TPDS.2021.3097283](https://doi.org/10.1109/TPDS.2021.3097283).
- [7] R. Reyes, Lomü, and V. Ller, “SYCL: Single-source C++ accelerator programming,” in *Parallel Computing: On the Road to Exascale*, IOS Press, 2016, pp. 673–682. doi: [10.3233/978-1-61499-621-7-673](https://doi.org/10.3233/978-1-61499-621-7-673).
- [8] C. Lattner et al., “MLIR: A Compiler Infrastructure for the End of Moore’s Law.” Accessed: Apr. 28, 2026. [Online]. Available: <http://arxiv.org/abs/2002.11054>
- [9] W. F. Godoy et al., “Mojo: MLIR-Based Performance-Portable HPC Science Kernels on GPUs for the Python Ecosystem,” in *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov. 2025, pp. 2114–2128. doi: [10.1145/3731599.3767573](https://doi.org/10.1145/3731599.3767573).
- [10] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis & transformation,” in *International symposium on code generation and optimization, 2004. CGO 2004.*, IEEE, 2004, pp. 75–86. Accessed: Apr. 24, 2026. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/1281665/>
- [11] J. E. Stone, D. Gohara, and G. Shi, “OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems,” *Computing in Science & Engineering*, vol. 12, no. 3, pp. 66–73, May 2010, doi: [10.1109/MCSE.2010.69](https://doi.org/10.1109/MCSE.2010.69).
- [12] “Mojo Manual | Mojo.” Accessed: May 08, 2026. [Online]. Available: <https://mojolang.org/docs/manual/>

- [13] P. D. Akre and U. Pacharaneey, “A Comprehensive Review of Mojo: A High-Performance Programming Language,” in *2025 6th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI)*, Jan. 2025, pp. 861–867. doi: [10.1109/ICMCSI64620.2025.10883176](https://doi.org/10.1109/ICMCSI64620.2025.10883176).